



gpp ya yandex,

English

Messages



minjem matering- Content

buat dikotret ∞ —, — ∞ ✓ Topic read

Problem statement. Least-squares method

In this module, we'll talk about selecting and training a model to solve a regression problem, taking a closer look at the linear regression model. We assume that the data we have for linear regression are **all in numerical form and represented by vectors of equal length**. That means we have N objects and n numerical features describing the objects, which we're going to use for predicting the target variable.

When talking about a **linear model**, we assume that our prediction is a weighted sum of features, which looks like this:

$$\hat{y} = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$$



x_1 fitur 1

w_0 bias

x_2 fitur 2

dst. x_n fitur n

In this formula, $x_1, \dots, x_n$ are feature values, $w_0, \dots, w_n$ are the weights we need to set, and $\hat{y}$ is the value we're predicting. To keep the notation simple, we add another feature x_0 that equals 1 for all elements.

Then, we can reduce the equation for each object by transforming it into vector form: $\hat{y} = \langle w \rangle$, where $\langle w \rangle$ is the scalar product of vectors $w = (w_0, \dots, w_n)$ and $x = (x_0, \dots, x_n)$.

If we factor in all objects, the model takes a matrix form:

$\hat{Y} = X \cdot w$, where X is an $N \times (n + 1)$ dimensional matrix, $\hat{Y}$ is an $N \times 1$ vector, and w is an $n \times 1$ vector.

nah? apaan N ? $\rightarrow$ bias gimana? $\rightarrow N \neq n$

Let's take a look at linear models using our real estate example. We want to build a model that predicts the price based on the area of an apartment. The model will look like this:

$\hat{y} = w_0 + w_1 * x_1$, where x_1 is the apartment area, and w_0 and w_1 are the weights we want to pick to more accurately guess this area. To keep it simple, for now we'll assume that $w_0 = 0$.

Obviously, the larger the apartment, the more expensive it is, so w_1 is a positive number.

$\rightarrow$ oo blm tentu bro, jk mempertimbangkan aspek selain area js.

Since we're solving a supervised learning problem, we already have a dataset ready with examples to train the model. But, as we know from the previous modules, we still need an error function that we're going to improve. Let's take the squared difference between the actual value and the predicted value as the error function:

$$E = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Least-squares
method



Here, N is the number of examples in the dataset, y_i is the target value of the i -th example, and $\hat{y}_i$ is the prediction of this target value. The closer $\hat{y}_i$ is to y , the smaller their difference and the magnitude of the error E . If $\hat{y}_i = y$ (that is, we accurately predicted the value), then the error is zero and does not increase E . We square the error so that negative and positive errors don't offset each other, and then divide the sum by the number of elements N because we want



to know the average error size. This metric is called the mean squared error (MSE), and we want to pick parameters for w in such a way that MSE has the least possible value.

That is: $E = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \rightarrow \min$, which is equivalent to $\sum_{i=1}^N (y_i - \hat{y}_i)^2 \rightarrow \min$.

Linear models have many benefits. For example, using linear models, it's quite easy to assess how certain features affect the output. Let's say, if the weight w_i is positive, then the value of the target variable will increase as the i -th feature gets bigger. Weight values are also straightforward in their interpretation: as the weight w_i increases, so does the importance of the i -th feature for the final prediction. You'll find it easy to explain the outputs of a linear model to a customer — this property of models is called interpretability. Interpretability is especially valued in industrial problems where the cost of error is high. If a person's life depends on the model's performance, it's essential to understand how the model makes decisions and what principles it follows. That said, not all machine learning methods are easily interpretable: for example, interpreting the behavior of artificial neural networks or gradient boosting is quite difficult.

make off using linear model
Nevertheless, you should not blindly trust the weights of a linear model. Keep in mind that these models are a rather narrow class of functions that perform well on small datasets and in solving simple problems. If you use a linear model for more complex tasks, you'll likely need to come up with additional features representing complex functions of the original features. And if you get too invested in finding such artificial features, it may unfortunately hinder the model's interpretability.

If there's an approximate linear dependence between the features, the coefficients in the linear model may completely lose physical meaning (we'll discuss this problem and how to deal with it later when we talk about regularization).

Be cautious about making claims like, "This coefficient is small, so this feature is not significant". First, the scale of the feature is important — perhaps the small coefficient is meant to compensate for it. Second, the dependence may actually be weak, but it may still become significant in certain situations.

Specific weight values may vary depending on the training dataset, although as the dataset grows in size, they'll gradually come closer to the weights of the most "optimal" linear model that could've been built using all available data in the world.

Next topic



Access to the service is provided by Y. Izdeu men Jarnama LLP. The service is not an educational service under the [Terms of Use](#).

© 2018 – 2026 Y. Izdeu men Jarnama LLP





Analytical solution

In the previous chapter, we established that we need to minimize the expression:

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 \rightarrow \min$$

If we replace $\hat{y}_i$ with $\langle w_i * x_i \rangle$, we get this:

$$\sum_{i=1}^N (y_i - \langle w_i * x_i \rangle)^2 \rightarrow \min$$

Here is the equivalent matrix notation of this expression:

$$(Y - Xw)^T (Y - Xw) \rightarrow \min$$

To verify the equivalence of the two expressions, you can simply expand the brackets. When checking, assume that $n = m = 2$ for simplicity.

Let's differentiate our expression with respect to w and equate it to zero to find the minimum. We get either this:

$$-2X^T(Y - Xw) = 0$$

Or this, which is exactly the same:

$$X^T(Y - Xw) = 0$$

Then if we expand the brackets and make a few simple transformations, we arrive at the solution:

$$\hat{w} = (X^T X)^{-1} X^T Y$$

In general, finding a solution is not always possible. For a solution to exist, the matrix $(X^T X)$ must be invertible, or its rank must equal $n + 1$ (which is the same thing).

Exact analytical solutions have one serious drawback — high computational complexity. The main problem is in the inversion of the matrix $(X^T X)$. Datasets in machine learning problems may contain millions of points, which renders them practically impossible to compute.

Moreover, if there's an approximate linear dependence between the features, a solution may be unstable, and small changes in y may lead to large changes in coefficients w .

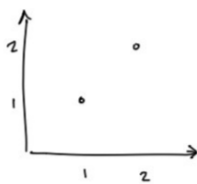
Next topic

urgensi pakai
ML y/ case linear
regression.

Access to the service is provided by Y. Izdeu men Jarnama LLP. The service is not an educational service under the [Terms of Use](#).

© 2018 – 2026 Y. Izdeu men Jarnama LLP

misal:



$y = ?$

$$\rightarrow \frac{y - y_1}{y_2 - y_1} = \frac{x - x_1}{x_2 - x_1}$$

$$\frac{y - 1}{2 - 1} = \frac{x - 1}{2 - 1}$$

$$y - 1 = x - 1$$

$$y = x$$

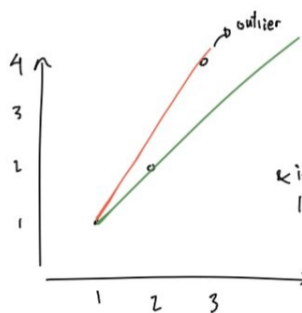
o-o

predict!

$$x = 3 \rightarrow y = ? \rightarrow y = 3 :))$$

0 error.

but.. how about this?:



it could be like that

$$\frac{y - y_1}{y_2 - y_1} = \frac{x - x_1}{x_2 - x_1}$$

$$\frac{y - 1}{4 - 1} = \frac{x - 1}{3 - 1}$$

$$\frac{y - 1}{3} = \frac{x - 1}{2}$$

$$y - 1 = (x - 1) \cdot \frac{3}{2}$$

$$y = \frac{3}{2}(x - 1) + 1$$

$$y = \frac{3}{2}x - \frac{3}{2} + 1$$

$$y = \frac{3}{2}x - \frac{1}{2}$$

$$\text{predict! } x = 2 \rightarrow y = ?$$

$$y = \frac{3}{2}(2) - \frac{1}{2}$$

$$= 3 - \frac{1}{2}$$

$$= 2\frac{1}{2}$$

$$\text{error} \neq 0, \text{ karena error} = \left| 2 - 2\frac{1}{2} \right| = \frac{1}{2}$$

& it could be worse if the outlier is more extreme than this.



Approximate solution. Gradient descent

Let's try to reduce the amount of calculations by replacing the exact solution with an approximate one. As we know, the function's gradient at a point is directed towards the fastest increase, while the anti-gradient is directed towards the decrease. If we take some random values of w_i , calculate the gradient at that point, and slightly shift the values toward the anti-gradient, we'll improve the value of the loss function.

$$w \rightarrow w - \alpha * \frac{2}{NX^T(Xw - y)}$$

$\alpha = \text{learning rate}$

$\text{rate } \downarrow = \text{min risk, underfitting}$

Here, $2X^T(Xw - y)$ is the gradient we calculated in the previous section, $1/N$ is the coefficient reducing the function to MSE, and α is the coefficient called the learning rate. The greater the α , the more the original weights are changed. Conversely, the smaller it is, the slower the change. Then we can re-measure the error function and repeat all the steps.

If α is too great, we risk missing the minimum and getting a solution that is too broad; if it's too small, more iterations are required to reach the minimum.

α is a hyperparameter that's set in advance and doesn't change after the training process starts, and w weights are the parameters that we pick during model training.

Gradient descent has a much lower computational complexity compared to an exact analytical solution, but it helps obtain a fairly accurate solution that is sufficient for most practical problems.

Next topic

Access to the service is provided by Y. Izdeu men Jarnama LLP. The service is not an educational service under the [Terms of Use](#).

© 2018 – 2026 Y. Izdeu men Jarnama LLP



Help

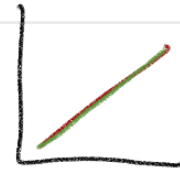


Regularization. L2, L1

Linear regression problems don't necessarily have a single solution. If the dataset contains two linearly dependent features, there must be a weight vector v for which $\langle w, x_i \rangle = 0$ for all x_i .

bergantung linear $w_1 x_1 + w_2 x_2 = 0$

$$w_1 x_1 = w_2 x_2$$



garis yg sama!

Linear dependence of features means that one feature can be expressed through a linear combination of other features. For example, if one feature is an exact copy of another feature or is the sum of that feature and some coefficient, then these features are linearly dependent. Mathematically, this is reflected in the **rank of the feature matrix X**: if the matrix rank is less than the number of features n , then there's a linear dependence between the features.

In this case, if $\hat{w}$ is a solution to the optimization problem, then $\hat{w} + av$ is also a solution for any a . So the solution is clearly not unique, and besides, it can be infinitely large by absolute value.

This leads to computational challenges:

- **Noise sensitivity:** weights that are large by absolute value make the model extremely sensitive to small changes in the input data. Minor inaccuracies or noise in features can lead to significant deviations in predictions.
- **Numerical instability:** when using optimization algorithms such as gradient descent, operations with very large numbers can cause the accumulation of errors due to the computer's limited computational accuracy.

Features with strict linear dependence are rare in real life, but they may well be approximately linearly dependent. This is called **multicollinearity**, and it brings about similar problems.

↳ approximately juga bisa jd masalah trngt yg

- **Instability of estimates:** weight estimates become unreliable and can change significantly with small changes in the data.
- **Loss of interpretability:** weights can take unexpected values and signs, making it more difficult to interpret the model.

multikolinearitas . how to avoid?

When several features are linearly dependent, the weights w_i attached to them lose physical meaning. It may even turn out that the weight of a feature, the increase in which is supposed to increase the target variable, becomes negative. This renders the model not only inaccurate but also uninterpretable. Irregular signs or weight values are good indications of multicollinearity.

this is how: ①

- ② The first thing to do in such cases is to remove one of the correlated features. In addition, you can apply **regularization** methods to the problem, which add an extra penalty for large weight values and help produce a more stable solution.

L1 and L2 regularizations are methods for adding a penalty to the loss function when machine learning models. These penalties control model complexity, prevent overfitting, and



improve the model's generalization ability. The methods borrow their names from functional analysis — they are actually L1 and L2 norms of the weight vector added to the loss function. With L2 regularization, we don't extract the square root as opposed to the norm.

L2 regularization supplements the loss function with a penalty proportional to the sum of the squared weights. For linear regression, the loss function with L2 regularization is expressed as follows:

$$\sum_{i=1}^N (y_i - \langle w_i * x_i \rangle)^2 + \alpha \sum_{j=0}^n (w_j)^2$$

With this loss function, we don't just aim for a small error size (difference between y and $y^{\wedge}$) but also for the minimum values of w .

Here we also add the α hyperparameter. It is set by the researcher and controls the influence of the weight sizes on the total penalty. The greater the α , the more the model tries to decrease the w_j weights and the simpler the model is, which means that it may end up being underfitted. On the contrary, the smaller the α , the less regularization affects the weights, and the chances of getting an overfitted or unstable model are greater.

L2 regularization penalizes large weights by forcing the "model" to distribute their effects across features, reducing and smoothing them out. In addition, L2 solves the problems associated with multicollinearity.

L1 regularization supplements the loss function with a penalty proportional to the sum of the weights' moduli:

$$\sum_{i=1}^N (y_i - \langle w_i * x_i \rangle)^2 + \alpha \sum_{j=0}^n |w_j|$$

The effect of L1 regularization is different from that of L2. Instead of evenly reducing the weight sizes, L1 nullifies some of the weights, selecting the most relevant ones. If we have several highly correlated features, it will select one and nullify the weights attached to the others. L1 regularization comes in handy in problems with a large number of features, where it's important to reduce the model complexity. The α hyperparameter is used here in a way similar to L2.

Let's imagine that there are two linearly correlated features in our data — for example, temperature in Celsius and Fahrenheit. L1 regularization will choose one of these features, whereas L2 will distribute the weights and try to make them smaller.

In real-life scenarios, L1 and L2 are sometimes combined; this type of regularization is called ElasticNet. We won't be covering this approach in the course, but we do recommend that you look into it.

Next topic



what is linear regression?

English

Messages



$$Y = ax + b ?$$

only a?

the coef?

Content

Topic read

$$Y = ax^2 + bx + c ?$$

→ no linear more?

or can it be like this too?

$$Y = ax + bx + c ?$$

Linear regression in sklearn

We'll now be using sklearn (Scikit-learn), which is a very handy ML library that has all the key machine learning models. We'll use our data to implement a linear regression model and make predictions with it. Let's assume we've already prepared the data for training.

First, we need to import the libraries:

```
1 import pandas as pd #for handling arrays and data tables
2 from sklearn.linear_model import LinearRegression #linear regression class
3 from sklearn.model_selection import train_test_split #function for splitting data
4 from sklearn.metrics import mean_squared_error, r2_score #metrics to assess the l
```

$Y = (a+b)x + c$ eh...
mx + c
oh!
see pg 1!

We assume that variable X contains the data and variable y contains the target feature.

Let's split our data into training and test datasets:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

```
1 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_s
```

Here, test_size=0.2 means that 20% of the data is used for testing, and random_state=42 sets the initial value of the random number generator. If we remove this parameter, we'll get a slightly different result each time because the starting weights will be initialized differently each time.

Now let's create an instance of the model and train it on the training data:

```
1 model = LinearRegression()
2 model.fit(X_train, y_train)
```

The fit method trains the model by fitting regression coefficients to the training data. That's it, done! We now have a trained model in the model variable. Simple, isn't it? To check out the predicted coefficients, execute the code:

Very simple :)

```
1 model.coef_
```

Now let's see if the model can make accurate productions on new data:

```
1 y_pred = model.predict(X_test)
```



Help

predict returns the predicted values of the target variable on the test data X_{test} . And because we know the actual values, we can see how far these predictions are from reality.

```
1 mse = mean_squared_error(y_test, y_pred)
```

`mean_squared_error` is the MSE parameter we discussed earlier that measures the mean square of the difference between actual and predicted values. As you know from the previous section, the smaller the MSE, the better the `model`.

Now we want to add regularization, which is also quite easy — just take a Lasso object for L1 regularization and a Ridge object for L2 regularization.

Next topic

Access to the service is provided by Y. Izdeu men Jarnama LLP. The service is not an educational service under the [Terms of Use](#).

© 2018 – 2026 Y. Izdeu men Jarnama LLP

? kalau fiturnya banyak, tetap bisa pakai linear regression? → ya,
see ps 1
? terus gimana ngitungnya?? → see ps 3-5